Approximate complex amplitude encoding algorithm and its application to data classification problems

Naoki Mitsuda,[1,2] Tatsuhiro Ichimura,[1,2] Kouhei Nakaji,[3,4,2] Yohichi Suzuki,[2] Tomoki Tanaka,[5,2] Rudy Raymond,[6,2,7] Hiroyuki Tezuka,[8,2,9] Tamiya Onodera,[6,2] and Naoki Yamamoto[2,10]

[1] *Sumitomo Mitsui Trust Bank, Ltd., 1-4-1, Marunouchi, Chiyoda-ku, Tokyo 100-8233, Japan*
[2] *Quantum Computing Center, Keio University, 3-14-1 Hiyoshi, Kohoku-ku, Yokohama, Kanagawa 223-8522, Japan*
[3] *Department of Chemistry, University of Toronto, Toronto, Ontario, Canada M5G 1Z8*
[4] *Research Center for Emerging Computing Technologies, National Institute of Advanced Industrial Science and Technology (AIST), 1-1-1 Umezono, Tsukuba, Ibaraki 305-8568, Japan.*
[5] *Mitsubishi UFJ Financial Group Inc. and MUFG Bank Ltd., 2-7-1 Marunouchi, Chiyoda-ku, Tokyo 100-8388*
[6] *IBM Quantum, IBM Japan, 19-21 Nihonbashi Hakozaki-cho, Chuo-ku, Tokyo 103-8510, Japan*
[7] *Department of Computer Science, University of Tokyo, 7-3-1 Hongo Bunkyo-ku, Tokyo 113-0033, Japan*
[8] *Advanced Research Laboratory, Technology Infrastructure Center, Technology Platform, Sony Group Corporation, 1-7-1 Konan, Minato-ku, Tokyo 108-0075, Japan*
[9] *Graduate School of Science and Technology, Keio University, 3-14-1 Hiyoshi, Kohoku-ku, Yokohama, Kanagawa 223- 8522, Japan*
[10] *Department of Applied Physics and Physico-Informatics, Keio University, Hiyoshi 3-14-1, Kohoku-ku, Yokohama, Kanagawa 223-8522, Japan*

arXiv:2211.13039v3 [quant-ph] 27 May 2024

Quantum computing has a potential to accelerate data processing efficiency, especially in machine learning, by exploiting special features such as the quantum interference. The major challenge in this application is that, in general, the task of loading a classical data vector into a quantum state requires an exponential number of quantum gates. The approximate amplitude encoding (AAE) method, which uses a variational means to approximately load a given real-valued data vector into the amplitude of a quantum state, was recently proposed as a general approach to this problem mainly for near-term devices. However, AAE cannot load a complex-valued data vector, which narrows its application range. In this work, we extend AAE so that it can handle a complex-valued data vector. The key idea is to employ the fidelity distance as a cost function for optimizing a parametrized quantum circuit, where the classical shadow technique is used to efficiently estimate the fidelity and its gradient. We apply this algorithm to realize the complex-valued-kernel binary classifier called the compact Hadamard classifier, and then we present a numerical experiment showing that it enables classification of the Iris dataset and credit card fraud detection.

I. INTRODUCTION

Quantum computing is expected to execute information processing tasks that classical computers cannot perform efficiently. One of the most promising domains in which quantum computing has a potential to boost its performance is machine learning [1–6]. The advantage of quantum computing in machine learning may come from the capability to represent and manipulate an exponential amount of classical data using quantum interference [7–9]. Furthermore, quantum computing can exponentially speed up basic linear algebra subroutines [10], which are at the core of quantum machine learning, such as the support vector machine [11] and the principal component analysis [12]. However, these applications require fault-tolerant quantum computers.

In recent years, some attempts to implement machine learning algorithms on shallow quantum circuits have been made, such as the SWAP-test classifier [13], the Hadamard test classifier (HTC) [14], and the compact Hadamard classifier (CHC) [15]. The central idea underlying these classifiers is that inner products in an exponentially large Hilbert space can be directly accessed by measurement without expensive subroutines [13, 14, 16]. It should be noted, however, that these classifiers assume that the classical data (i.e., the training data and the test data) have been loaded into the amplitudes of a quantum state, i.e., *amplitude encoding*.

In general, the number of gates exponentially grows with the number of qubits for realizing the amplitude encoding [17–23], which might be a major bottleneck in practical applications of quantum computation. In order to address this issue, Ref. [24] proposed an algorithm called the *approximate amplitude encoding (AAE)* that generates approximated n-qubit quantum states using $\mathcal{O}(\mathrm{poly}(n))$ gates. However, AAE is only applicable to the problem of loading a real-valued data vector; that is, it cannot load a complex-valued data vector. This limitation narrows the scope of AAE application. For example, AAE cannot be applied for preparing an initial state of the CHC [15], because the CHC encodes the data into both the real and imaginary parts of the amplitude of the underlying quantum state. Aside from the CHC, an efficient complex-valued data encoding method is also required for state preparation of wavepacket dynamical simulations in quantum chemistry [25, 26].

In this paper, we extend the AAE method so that it can approximately load a complex-valued data vector onto a shallow quantum circuit. We refer to this algorithm as *approximate complex amplitude encoding (ACAE)*. The key idea is to change the cost function from the maximum mean discrepancy (MMD) [27, 28], used in AAE, to the fidelity, which can capture the difference in complex amplitude between two quantum states unlike the MMD-based cost function. A notable point is that the *classical shadow* [29] is used to efficiently estimate the fidelity or its gradient. As a result, ACAE enables embedding the real and imaginary parts of any complex-valued data vector into the amplitudes of a quantum state. In addition, we provide an algorithm composed of ACAE and the CHC; this algorithm realizes a quantum circuit for binary data classification, using fewer gates than the original CHC which requires exponentially many gates to prepare the exact initial quantum state. We then give a proof-of-principle demonstration of this classification algorithm for the benchmark Iris data classification problem. Moreover, we apply the algorithm to the credit card fraud detection problem, which is a key challenge in financial institutions.

II. APPROXIMATE COMPLEX AMPLITUDE ENCODING ALGORITHM

A. Goal of the approximate complex amplitude encoding algorithm

Quantum state preparation is an important subroutine in quantum algorithms that process classical data. Ideally, this subroutine is represented by a state preparation oracle, U, that encodes an N-dimensional complex vector, $\boldsymbol{c} = \{c_0, \ldots, c_{N-1}\}, c_k \in \mathbb{C}$, to the amplitudes of an n-qubit state, $|\mathrm{Data}\rangle$:

$$|\mathrm{Data}\rangle = U|0\rangle^{\otimes n} = \sum_{k=0}^{N-1} c_k |k\rangle, \tag{1}$$

where the input vector is normalized as $\|\boldsymbol{c}\| = 1$. Note that $n = \lceil \log_2(N) \rceil$. Hereafter, the state (1) is referred to as the *target state*. Recall that, in general, a quantum circuit for generating the target state requires $O(2^n)$ controlled-NOT (CNOT) gates [17–23], which might destroy any possible quantum advantage.

The objective of ACAE is to generate a quantum state that approximates the target state (1), using a parametrized quantum circuit (PQC) that is represented by the unitary matrix $U(\boldsymbol{\theta})$, with $\boldsymbol{\theta}$ being the vector of parameters. Hereafter, we refer to the state generated from the PQC as a *model state*, i.e., $|\psi(\boldsymbol{\theta})\rangle = U(\boldsymbol{\theta})|0\rangle^{\otimes n}$. We train a PQC to approximate the target state except for the global phase; hence, ideally, the model state is trained to satisfy

$$|\psi(\boldsymbol{\theta})\rangle = U(\boldsymbol{\theta})|0\rangle^{\otimes n} = e^{i\alpha}|\mathrm{Data}\rangle, \tag{2}$$

where $e^{i\alpha}$ is the global phase.

Note that, if the elements of $\boldsymbol{c} = \{c_0, \ldots, c_{N-1}\}$ are all real-numbers, we can use AAE [24], which also trains a PQC to generate an approximating state. The training is accomplished by minimizing the cost function given by the MMD [27, 28] between two probability distributions corresponding to the target and the model states. However, this method is not applicable for the general complex-valued data loading, because each element of the probability distribution is given by the squared absolute value of the complex amplitude of the corresponding state vector and thus AAE cannot distinguish real and complex numbers of the state.

B. The proposed algorithm

1. Cost function

In order to execute a complex-valued data loading, it is necessary to introduce a measure that reflects the difference between two quantum states with complex-valued amplitudes, which cannot be captured by the MMD-based cost function. Here we employ the fidelity between the model state $|\psi(\boldsymbol{\theta})\rangle$ and the target state $|\mathrm{Data}\rangle$:

$$f(\boldsymbol{\theta}) = \mathrm{Tr}\left(\rho_{\mathrm{model}}(\boldsymbol{\theta})\rho_{\mathrm{target}}\right) = |\langle \mathrm{Data}|\psi(\boldsymbol{\theta})\rangle|^2, \tag{3}$$

where $\rho_{\mathrm{model}}(\boldsymbol{\theta}) = |\psi(\boldsymbol{\theta})\rangle\langle\psi(\boldsymbol{\theta})|$ and $\rho_{\mathrm{target}} = |\mathrm{Data}\rangle\langle\mathrm{Data}|$. Although, in general, the fidelity can be estimated using the quantum state tomography [30], it is highly resource-intensive because this procedure requires accurate expectation values for a set of observables whose size grows exponentially with respect to the number of qubits. For this reason, we employ the classical shadow technique to estimate the fidelity.

2. Fidelity estimation by classical shadow

Classical shadow [29] is a method for constructing a classical representation that approximates a quantum state using much fewer measurements than the case of state tomography. The general goal is to predict the expectation values o_j for a set of L observables, O_j:

$$o_j(\rho) = \mathrm{Tr}\left(O_j \rho\right), \ 1 \le j \le L, \tag{4}$$

where ρ is the underlying density matrix. The procedure for constructing the predictor is described below.

First, ρ is transformed by the unitary operator U taken from the set of random unitaries $\mathcal{U}$ as $\rho \to U\rho U^\dagger$, and then each qubit is measured in the computational basis. For a measurement outcome, $|\hat{b}\rangle$, the reverse operation $U^\dagger|\hat{b}\rangle\langle\hat{b}|U$ is calculated and stored in a classical memory. The averaging operation on $U^\dagger|\hat{b}\rangle\langle\hat{b}|U$ with respect to $U \in \mathcal{U}$ is regarded as a quantum channel on ρ;

$$\mathbb{E}\left[U^\dagger|\hat{b}\rangle\langle\hat{b}|U\right] = \mathcal{M}(\rho), \tag{5}$$

which implies

$$\rho = \mathbb{E}\left[\mathcal{M}^{-1}\left(U^\dagger|\hat{b}\rangle\langle\hat{b}|U\right)\right]. \tag{6}$$

The quantum channel $\mathcal{M}$ depends on the ensemble of unitary transformations $\mathcal{U}$. Equation (6) gives us a procedure for constructing an approximator for ρ. That is, if the above measurement plus reverse operation is performed N_{shot} times for different $U \in \mathcal{U}$, then we obtain an array of N_{shot}-independent classical snapshots of ρ:

$$\begin{aligned}\mathsf{S}(\rho; N_{\text{shot}}) = \Big\{\hat{\rho}_1 = \mathcal{M}^{-1}\left(U_1^\dagger\,|\hat{b}_1\rangle\langle\hat{b}_1|\,U_1\right), \ldots, \\ \hat{\rho}_{N_{\text{shot}}} = \mathcal{M}^{-1}\left(U_{N_{\text{shot}}}^\dagger\,|\hat{b}_{N_{\text{shot}}}\rangle\langle\hat{b}_{N_{\text{shot}}}|\,U_{N_{\text{shot}}}\right)\Big\}.\end{aligned} \tag{7}$$

This array is called the classical shadow of ρ. Once a classical shadow (7) is obtained, an estimator of $\hat{o}_j$ can be calculated as

$$\hat{o}_j(\rho) = \frac{1}{N_{\text{shot}}}\sum_{i=1}^{N_{\text{shot}}} \text{Tr}\left(O_j\hat{\rho}_i\right), \tag{8}$$

where each $\hat{\rho}_i$ is the classical snapshot in $\mathsf{S}(\rho; N_{\text{shot}})$. Although Reference [29] proposed to use the median-of-means estimator, we employ the empirical mean for simplicity of the implementation. Ref. [29] proved that this protocol has the following sampling complexity.

Theorem [29]. *Classical shadows of size N_{shot} suffice to predict L arbitrary linear target functions $\text{Tr}(O_1\rho), \ldots, \text{Tr}(O_L\rho)$ up to additive error ϵ given that*

$$N_{\text{shot}} \geq \mathcal{O}\left(\frac{\log(L)}{\epsilon^2}\max_j \|O_j\|^2_{\text{shadow}}\right). \tag{9}$$

The definition of the shadow norm $\|O_j\|_{\text{shadow}}$ depends on the ensemble $\mathcal{U}$. Two different ensembles can be considered for selecting the random unitaries U:

- random Clifford measurements, U belongs to the n-qubit Clifford group; and
- random Pauli measurements, each U is a tensor product of single-qubit operations.

For the random Clifford measurements, $\|O\|^2_{\text{shadow}}$ is closely related to the Hilbert-Schmidt norm $\text{Tr}(O^2)$. As a result, a large collection of (global) observables with bounded Hilbert-Schmidt norm can be predicted efficiently. For the random Pauli measurements, on the other hand, the shadow norm scales exponentially in the locality of the observable; note that, for certain cases of the random Pauli measurements, we can use the decision-diagram-based classical shadow to have an efficient measurement scheme [31].

The above classical shadow technique can be directly applied to the problem of estimating the fidelity $f(\boldsymbol{\theta})$ given in Eq. (3). Actually this corresponds to $L = 1$, $O = O_1 = \rho_{\text{target}}$, and $\rho = \rho_{\text{model}}(\boldsymbol{\theta})$ in Eq. (4); then, from Eq. (8), we have the estimate of $f(\boldsymbol{\theta})$ as

$$\hat{f}(\boldsymbol{\theta}) = \frac{1}{N_{\text{shot}}}\sum_{i=1}^{N_{\text{shot}}} \text{Tr}\left(O\hat{\rho}_i(\boldsymbol{\theta})\right), \tag{10}$$

where $\hat{\rho}_i(\boldsymbol{\theta})$ is the classical snapshot of $\rho_{\text{model}}(\boldsymbol{\theta})$. In this case, the random Clifford measurements should be selected, because the shadow norm is given by $\text{Tr}(\rho^2_{\text{target}}) = 1$; also, N_{shot} becomes independent of system size, because the $\max_j \|O_j\|^2_{\text{shadow}}$ term now becomes constant. For the random Clifford measurements, Reference [29] shows that the inverted quantum channel $\mathcal{M}^{-1}$ is given by

$$\mathcal{M}^{-1}(\rho) = (2^n + 1)\rho - I. \tag{11}$$

Lastly note that $\mathcal{O}\left(n^2/\log_2(n)\right)$ entangling gates are needed to do sampling from the n-qubit Clifford unitaries [32, 33], which is a practical drawback.

3. *Optimization of $U(\boldsymbol{\theta})$*

Here we describe the training method of the PQC for ACAE. The PQC $U(\boldsymbol{\theta})$ consists of n qubits and l layers; thus $U(\boldsymbol{\theta})$ contains $\mathcal{O}(ln)$ gates. The number of layers l is of the order from $\mathcal{O}(1)$ to $\mathcal{O}(\text{poly}(n))$. In this paper we take the PQC composed of single-qubit parametrized rotational gates $R_x(\theta_r) = \exp\left(-i\theta_r\sigma_x/2\right)$, $R_y(\theta_r) = \exp\left(-i\theta_r\sigma_y/2\right)$, and $R_z(\theta_r) = \exp\left(-i\theta_r\sigma_z/2\right)$ together with CNOT gates; here θ_r is the r-th element of $\boldsymbol{\theta}$ and σ_x, σ_y, and σ_z are the Pauli X, Y, and Z operators, respectively. We take the so-called hardware efficient ansatz [34]; an example of the structure is shown in Fig. 1. This PQC is followed by a random Clifford unitary U_i as shown in Fig. 2. The output of the circuit is measured N_{shot} times in the computational basis, with changing the random Clifford unitary U_i and obtaining the outcome $\hat{b}_i$ in each trial. The above procedure provides us with the classical snapshots of ρ_{model}. Note that each $|\hat{b}_i\rangle$ and U_i can be stored efficiently in a classical memory using the stabilizer formalism [32].

Our goal is to find the best $\boldsymbol{\theta}$ that maximizes the fidelity $f(\boldsymbol{\theta})$. Now we use the classical shadow to have the estimate $\hat{f}(\boldsymbol{\theta})$ given in Eq. (10), which can be further calculated as

$$\begin{aligned}\hat{f}(\boldsymbol{\theta}) &= \frac{1}{N_{\text{shot}}}\sum_{i=1}^{N_{\text{shot}}} \text{Tr}\left(O\hat{\rho}_i(\boldsymbol{\theta})\right) \\ &= \frac{1}{N_{\text{shot}}}\sum_{i=1}^{N_{\text{shot}}} \text{Tr}\left[O\mathcal{M}^{-1}\left(U_i^\dagger\,|\hat{b}_i\rangle\langle\hat{b}_i|\,U_i\right)\right] \\ &= \frac{1}{N_{\text{shot}}}\sum_{i=1}^{N_{\text{shot}}} \text{Tr}\left[O\left\{(2^n+1)U_i^\dagger\,|\hat{b}_i\rangle\langle\hat{b}_i|\,U_i - I\right\}\right] \\ &= \frac{1}{N_{\text{shot}}}\sum_{i=1}^{N_{\text{shot}}} (2^n+1)\,\langle\hat{b}_i|\,U_i O U_i^\dagger\,|\hat{b}_i\rangle - 1.\end{aligned}$$

Let us consider the run time for evaluating the above $\hat{f}(\boldsymbol{\theta})$ with a classical computer. First, each $\langle\hat{b}_i|\,U_i O U_i^\dagger\,|\hat{b}_i\rangle$ is

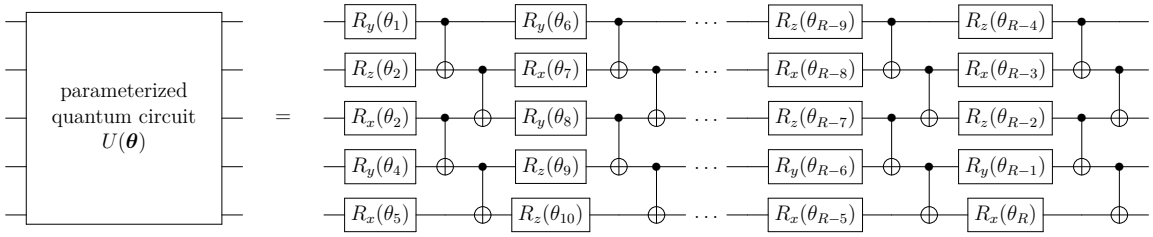


FIG. 1. Example of the structure of the hardware efficient ansatz $U(\boldsymbol{\theta})$. Each layer is composed of the set of parametrized single-qubit rotational gates $R_x(\theta_r) = \exp(-i\theta_r\sigma_x/2), R_y(\theta_r) = \exp(-i\theta_r\sigma_y/2)$, or $R_z(\theta_r) = \exp(-i\theta_r\sigma_z/2)$. We randomly initialize all axes of each rotating gate (i.e., X, Y, or Z) and θ_r at the beginning of each training.

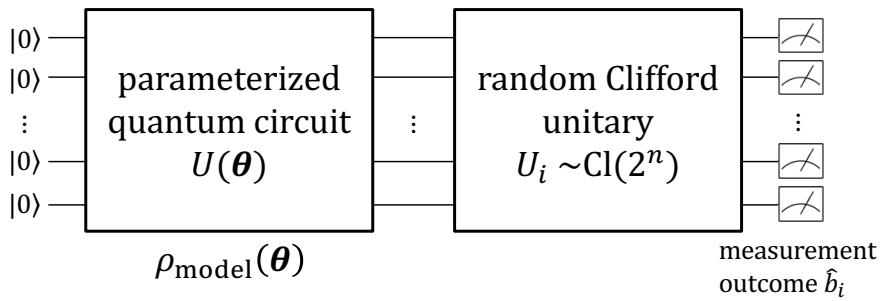


FIG. 2. Configuration of the n-qubit quantum circuit for training PQC $U(\boldsymbol{\theta})$. The PQC is followed by a unitary, U_i, which is selected from the random Clifford unitaries with the size 2^n. Measurement is carried out to obtain the outcome $\hat{b}_i$, i.e., a bit string of length n.

calculated as follows:

$$\begin{aligned}\langle \hat{b}_i | U_i O U_i^\dagger | \hat{b}_i \rangle &= \langle \hat{b}_i | U_i | \mathrm{Data} \rangle \langle \mathrm{Data} | U_i^\dagger | \hat{b}_i \rangle \\ &= \left| \langle \hat{b}_i | U_i | \mathrm{Data} \rangle \right|^2 \\ &= \left| \sum_{k=0}^{N-1} c_k \langle \hat{b}_i | U_i | k \rangle \right|^2 . \end{aligned} \tag{12}$$

The Gottesman-Knill theorem [35] allows for evaluation of $\langle \hat{b}_i | U_i | k \rangle$ in $\mathcal{O}(n^2)$ time, because $(|\hat{b}_i\rangle, |k\rangle)$ and U_i are stabilizer states and a Clifford operator, respectively. Note that the summation in Eq. (12) requires $\mathcal{O}(N) = \mathcal{O}(2^n)$ computations, which means that the required run time in the training process of the PQC scales exponentially with the number of qubits. However, this is *classical* computation, which should become exponential as long as we would like to process a general exponential-size classical data. Rather, the advantage of ACAE is in the depth of the PQC, which operates only $\mathcal{O}(n\,\mathrm{poly}(n))$ gates instead of $\mathcal{O}(2^n)$, to achieve $\mathcal{O}(2^n)$ data encoding.

To maximize the fidelity estimate $\hat{f}(\boldsymbol{\theta})$ [i.e., to minimize the $-\hat{f}(\boldsymbol{\theta})$)], we take the standard gradient descent algorithm. The gradients of $\hat{f}(\boldsymbol{\theta})$ with respect to θ_r can be computed by using the parameter shift rule [36] as

$$\frac{\partial \hat{f}(\boldsymbol{\theta})}{\partial \theta_r} = \hat{f}^+_{\theta_r} - \hat{f}^-_{\theta_r}, \tag{13}$$

where

$$\hat{f}^\pm_{\theta_r} = \hat{f}\left(\theta_1, \cdots, \theta_{r-1}, \theta_r \pm \pi/2, \theta_{r+1}, \cdots, \theta_R\right). \tag{14}$$

R denotes the number of the parameters, which can be written as $R = ln$ (recall that l is the number of layers of PQC). That is, the gradient can also be effectively estimated using the classical shadow. This maximization procedure will ideally bring us the optimal parameter set $\boldsymbol{\theta}^*$ and unitary $U(\boldsymbol{\theta}^*)$ that generates a state approximating the target state (2).

III. APPLICATION TO COMPACT HADAMARD CLASSIFIER

This section first reviews the method of compact amplitude encoding and the compact Hadamard classifier (CHC). Then we describe how to apply ACAE to implement the CHC.

A. Compact amplitude encoding

This method encodes two real-valued data vectors into real and imaginary parts of the amplitude of a single quantum state. More specifically, given two N-dimensional real-valued vectors $\boldsymbol{x}_j^+ = (x_{0j}^+, \ldots, x_{(N-1)j}^+)^T$ and $\boldsymbol{x}_j^- = (x_{0j}^-, \ldots, x_{(N-1)j}^-)^T$, the

compact amplitude encoder prepares the following quantum state:

$$|\boldsymbol{x}_j\rangle := \sum_{l=0}^{N-1} (x_{lj}^+ + i x_{lj}^-)\,|l\rangle\,, \tag{15}$$

where

$$\|\boldsymbol{x}_j^+\|^2 + \|\boldsymbol{x}_j^-\|^2 = 1 \tag{16}$$

is assumed to satisfy the normalization condition. For simplicity, here we assume $\|\boldsymbol{x}_j^\pm\| = 1/\sqrt{2}$ without loss of generality. In addition, we define $|\boldsymbol{x}_j^\pm\rangle$ as

$$|\boldsymbol{x}_j^\pm\rangle := \frac{1}{\|\boldsymbol{x}_j^\pm\|}\sum_{l=0}^{N-1} x_{lj}^\pm\,|l\rangle = \sqrt{2}\sum_{l=0}^{N-1} x_{lj}^\pm\,|l\rangle\,. \tag{17}$$

B. Compact Hadamard classifier

Here we give a quick review about CHC [15]. Suppose the following training data set $\mathcal{D}$ is given as

$$\mathcal{D} = \{(\boldsymbol{x}_0, y_0)\,, \ldots, (\boldsymbol{x}_{M-1}, y_{M-1})\}\,.$$

All inputs $\{\boldsymbol{x}_j\}$ are N-dimensional real-valued vectors, and each y_j takes either $+1$ or -1. The goal of the CHC is to predict the label $\tilde{y}$ for a test datum $\tilde{\boldsymbol{x}}$, which is also an N-dimensional real-valued vector. For simplicity, we assume that the number of training data with label $+1$, denoted by M^+, is equal to the number of training data with label -1, denoted by M^-; i.e., $M^+ = M^- = M/2$, where M is an even number. In particular, we sort the training data set so that

$$\boldsymbol{x}_j = \begin{cases} \boldsymbol{x}_j^+ & (0 \le j \le M/2-1), \\ \boldsymbol{x}_{j-M/2}^- & (M/2 \le j \le M-1), \end{cases}$$

and

$$y_j = \begin{cases} +1 & (0 \le j \le M/2-1), \\ -1 & (M/2 \le j \le M-1). \end{cases}$$

Note that the CHC can also be applied to imbalanced training data sets as we see later.

Assuming the existence of the compact amplitude encoder $U_{\mathrm{CAE}}(\boldsymbol{x}_j)$ that encodes the two training data vectors $\boldsymbol{x}_j^\pm$ into a single quantum state (15) and the encoder $U_{\mathrm{AE}}(\tilde{\boldsymbol{x}})$ that encodes a test data vector $\tilde{\boldsymbol{x}}$ into the quantum state $|\tilde{\boldsymbol{x}}\rangle$, the following quantum state can be generated:

$$\begin{aligned} |\psi_{\mathrm{init}}\rangle &= U_{\mathrm{AE}}(\tilde{\boldsymbol{x}}) U_{\mathrm{CAE}}(\boldsymbol{x}_j) R_z^{(\mathrm{A})} H^{(\mathrm{A})} U_w^{(\mathrm{J})}(b)\,|0\rangle_{\mathrm{A}}\,|0\rangle_{\mathrm{D}}\,|0\rangle_{\mathrm{J}}\,, \\ &= U_{\mathrm{AE}}(\tilde{\boldsymbol{x}}) U_{\mathrm{CAE}}(\boldsymbol{x}_j) \sum_{j=0}^{\frac{M}{2}-1} \sqrt{b_j}\,\frac{|0\rangle_{\mathrm{A}} + e^{-i\phi}\,|1\rangle_{\mathrm{A}}}{\sqrt{2}}\,|0\rangle_{\mathrm{D}}\,|j\rangle_{\mathrm{J}}\,, \\ &= \frac{1}{\sqrt{2}}\sum_{j=0}^{\frac{M}{2}-1} \sqrt{b_j}\left(|0\rangle_{\mathrm{A}}\,|\boldsymbol{x}_j\rangle_{\mathrm{D}} + e^{-i\phi}\,|1\rangle_{\mathrm{A}}\,|\tilde{\boldsymbol{x}}\rangle_{\mathrm{D}}\right)|j\rangle_{\mathrm{J}}\,, \end{aligned} \tag{18}$$

where ϕ is the relative phase. Also, b_j is the set of weights satisfying $\sum_{j=0}^{M/2-1} b_j = 1$; in this work, we follow Reference [15] to choose the uniform weighting $b_j = 2/M$. The labels A, J, and D mean the ancilla qubit, the qubits for data numbering, and the qubits for data encoding, respectively (the subscript is omitted unless misunderstood). The number of qubits required to prepare this state is $n+m+1$, where $n = \lceil \log_2(N) \rceil$ and $m = \lceil \log_2(M/2) \rceil$. Then, we operate the single-qubit Hadamard gate on the ancilla qubit and get

$$\begin{aligned} |\psi_f\rangle := H^{(\mathrm{A})}\,|\psi_{\mathrm{init}}\rangle = &\frac{1}{2}\sum_{j=0}^{\frac{M}{2}-1} \sqrt{b_j}\big[\,|0\rangle_{\mathrm{A}}\,(|\boldsymbol{x}_j\rangle_{\mathrm{D}} + e^{-i\phi}\,|\tilde{\boldsymbol{x}}\rangle_{\mathrm{D}}) \\ &+ |1\rangle_{\mathrm{A}}\,(|\boldsymbol{x}_j\rangle_{\mathrm{D}} - e^{-i\phi}\,|\tilde{\boldsymbol{x}}\rangle_{\mathrm{D}})\big]\,|j\rangle_{\mathrm{J}}\,. \end{aligned} \tag{19}$$

Finally, we measure the ancilla qubit in the computational basis. The entire quantum circuit is illustrated in Fig. 3.

Now, the probabilities that the measurement outcome of the ancilla qubit is $|0\rangle$ and $|1\rangle$ are given by

$$\Pr(0) = \frac{1}{2}\sum_{j=0}^{\frac{M}{2}-1} b_j\,(1 + \cos(\phi)\mathrm{Re}(\kappa_j) - \sin(\phi)\mathrm{Im}(\kappa_j))$$

and $\Pr(1) = 1 - \Pr(0)$, where $\kappa_j = \langle\tilde{\boldsymbol{x}}|\boldsymbol{x}_j\rangle$. Therefore, the expectation value of the Pauli Z operator measured on the ancilla qubit, denoted as $\sigma_z^{(\mathrm{A})}$, is

$$\langle\sigma_z^{(\mathrm{A})}\rangle = \sum_{j=0}^{M/2-1} b_j\,(\cos(\phi)\mathrm{Re}(\kappa_j) - \sin(\phi)\mathrm{Im}(\kappa_j))\,.$$

If we set $\phi = \pi/4$, this becomes

$$\langle\sigma_z^{(\mathrm{A})}\rangle = \frac{1}{2}\sum_{j=0}^{\frac{M}{2}-1} b_j\left(\langle\tilde{\boldsymbol{x}}|\boldsymbol{x}_j^+\rangle - \langle\tilde{\boldsymbol{x}}|\boldsymbol{x}_j^-\rangle\right). \tag{20}$$

Note that, when the number of training data vectors in the two classes are not equal (i.e., $M^+ \neq M^-$), this difference can be compensated by tuning ϕ to satisfy $\tan(\phi) = M_-/M_+$. We now end up with the final form of Eq. (20) as

$$\langle\sigma_z^{(\mathrm{A})}\rangle = \frac{1}{2}\sum_{j=0}^{M-1} b'_j y_j \langle\tilde{\boldsymbol{x}}|\boldsymbol{x}_j\rangle, \tag{21}$$

where b'_j is defined as

$$b'_j = b'_{j+M/2} = b_j, \quad \sum_{j=0}^{M-1} b'_j = 2.$$

Clearly Eq. (21) has the form of a standard kernel-based classifier where $\langle\tilde{\boldsymbol{x}}|\boldsymbol{x}_j\rangle$ represents the similarity between the test data $\tilde{\boldsymbol{x}}$ and the training data $\boldsymbol{x}_j$. Because the

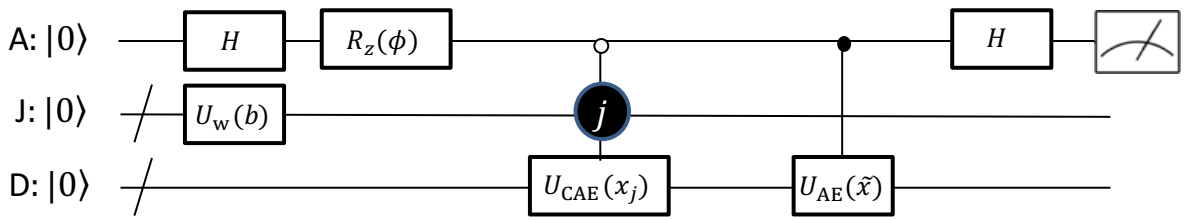


FIG. 3. Configuration of the circuit for the compact Hadamard classifier. The labels A, J, and D on the left side mean the ancilla qubit, the qubits for data numbering, and the qubits for data encoding, respectively. Slash symbols on the J and D lines indicate that the line is composed of multiple qubits. The number of qubits of the J and D lines are $m = \lceil \log_2(M/2) \rceil$ and $n = \lceil \log_2(N) \rceil$, where M and N represent the number of training data and the dimension of the data vector, respectively. $U_{\mathrm{w}}(b)$ is an operator that weights each $|j\rangle$ by b_j. $U_{\mathrm{CAE}}(\boldsymbol{x}_j)$ and $U_{\mathrm{AE}}(\tilde{\boldsymbol{x}})$ are the unitary operators that encode the training data $\boldsymbol{x}_j^{\pm}$ and the test data $\tilde{\boldsymbol{x}}$ into $|\boldsymbol{x}_j\rangle_{\mathrm{D}}$ and $|\tilde{\boldsymbol{x}}\rangle_{\mathrm{D}}$ in Eq. (18), respectively. The white and black dots connected to $U_{\mathrm{CAE}}(\boldsymbol{x}_j)$ and $U_{\mathrm{AE}}(\tilde{\boldsymbol{x}})$ indicate that the control action turns on when the variable takes 1 and 0, respectively. The black circle marked J indicates that the unitary operation $U_{\mathrm{CAE}}(\boldsymbol{x}_j)$ is the uniformly controlled via the J qubits in addition to the A qubit.

right-hand side of Eq. (21) represents the sum of the kernel weighted by $b'_j y_j$, the sign of $\langle \sigma_z^{(\mathrm{A})} \rangle$ tells us the class for the test data $\tilde{\boldsymbol{x}}$; that is, the CHC predicts the label $\tilde{y}$ of $\tilde{\boldsymbol{x}}$ via the following policy:

$$\tilde{y} = \mathrm{sgn}\left[\langle \sigma_z^{(\mathrm{A})} \rangle\right] = \mathrm{sgn}\left[\frac{1}{2}\sum_{j=0}^{M-1} b'_j y_j \langle \tilde{\boldsymbol{x}} | \boldsymbol{x}_j \rangle\right]. \tag{22}$$

Note that the weights $\{b'_j\}$ can be optimized, like the standard kernel-based classifier such as the support vector machine which indeed optimizes those weights depending on the training dataset; this clearly improves the classification performance of ACAE, which will be examined in a future work.

The advantage of quantum kernel-based classifiers over classical classifiers is the accessibility to kernel functions. Actually, the kernel (or similarity) between test data and training data is calculated as the inner product in the feature Hilbert space, which is computationally expensive to evaluate via classical means when the feature space is large. On the other hand, quantum kernel-based classifiers efficiently evaluate kernel functions. In particular, the CHC can evaluate the sum of all the inner products in the N-dimensional feature space appearing in the right-hand side of Eq. (21), just by measuring the expected value of $\sigma_z^{(\mathrm{A})}$. We also emphasize that the CHC can be realized with compact quantum circuits compared with other quantum kernel-based classifiers. Actually, thanks to the compact amplitude encoding, two qubits can be removed in the CHC formulation compared to the others; moreover, the number of operations for encoding the training data set $\boldsymbol{x}_j$ is reduced by a factor of 4 compared with the HTC [14]. Hence, the CHC can be implemented in a compact quantum circuit in both depth and width compared to the other quantum classifiers, meaning that a smaller and thus easier-trainable variational circuit may function for the CHC.

C. Implementation of the CHC using ACAE

Although the CHC efficiently realizes a compact quantum classifier, it relies on the critical assumption; that is, the quantum state (15) is necessarily prepared. Recall that, in general, the quantum circuit for generating the state (15) requires an exponential number of gates. Moreover, to generate the quantum state $|\psi_{\mathrm{init}}\rangle$ in Eq. (18), the uniformly controlled gate [37, 38] shown in Fig. 3 also requires an exponential number of gates. These requirements may destroy the quantum advantage of the CHC.

The ACAE implements the CHC without using exponentially many gates; that is, as shown below, we can approximately generate the quantum state $|\psi_f\rangle$ with a constant-depth quantum circuit illustrated in Fig. 4. First, by applying $H, R_z(\phi)$, and $U_w(b)$ on the initial states, we produce the following state:

$$|\psi_0\rangle := \frac{1}{\sqrt{2}} \sum_{j=0}^{\frac{M}{2}-1} \sqrt{b_j} \left(|0\rangle_{\mathrm{A}} |0\rangle_{\mathrm{D}} + e^{-i\phi} |1\rangle_{\mathrm{A}} |0\rangle_{\mathrm{D}} \right) |j\rangle_{\mathrm{J}} . \tag{23}$$

Next, we train $U(\boldsymbol{\theta})$ by the algorithm described in Sec. II, so that it approximately encodes the training data $\{\boldsymbol{x}_j\}$ to have

$$\begin{aligned} |\psi_1\rangle &:= U(\boldsymbol{\theta}) |\psi_0\rangle \\ &\approx e^{i\alpha} \frac{1}{\sqrt{2}} \sum_{j=0}^{\frac{M}{2}-1} \sqrt{b_j} \left(|0\rangle_{\mathrm{A}} |\boldsymbol{x}_j\rangle_{\mathrm{D}} + e^{-i\phi} |1\rangle_{\mathrm{A}} |0\rangle_{\mathrm{D}} \right) |j\rangle_{\mathrm{J}} , \end{aligned} \tag{24}$$

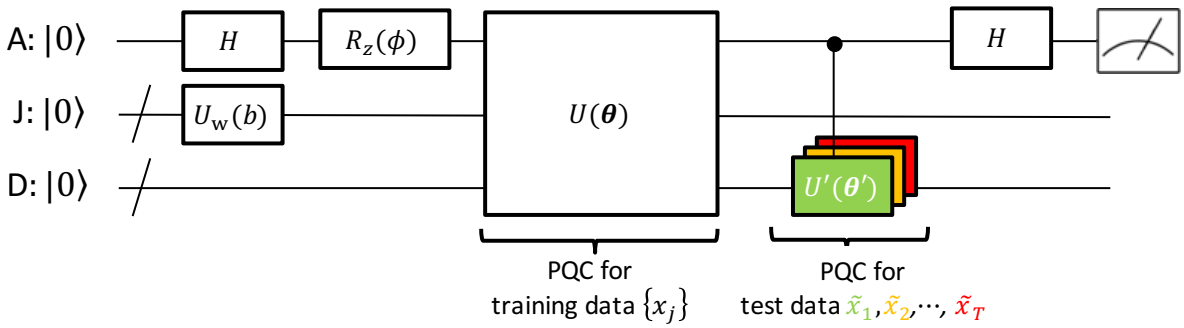


FIG. 4. Configuration of the circuit for the compact Hadamard classifier using ACAE. We use PQCs instead of the uniformly controlled $U_{\mathrm{CAE}}(\boldsymbol{x}_j)$ and $U_{\mathrm{AE}}(\tilde{\boldsymbol{x}})$ in Fig. 3; that is, $U(\boldsymbol{\theta})$ represents a PQC for encoding the training data $\{\boldsymbol{x}_j\}$ and thereby approximately generates $|\boldsymbol{x}_j\rangle_{\mathrm{D}}$ in Eq. (19). Also, $U'(\boldsymbol{\theta'})$ represents a PQC for encoding the test data $\tilde{\boldsymbol{x}}$ and thereby approximately generates $|\tilde{\boldsymbol{x}}\rangle_{\mathrm{D}}$. Note that, in practice, we construct $U(\boldsymbol{\theta})$ off-line using all the training data; then we learn $U'(\boldsymbol{\theta'})$ for a given test datum $\tilde{\boldsymbol{x}}$ for the on-line prediction purpose.

where $e^{i\alpha}$ is the global phase. We then set the controlled $U'(\boldsymbol{\theta'})$ to encode the test data $\tilde{\boldsymbol{x}}$ to obtain

$$\begin{aligned}
&\left(|0\rangle_{\mathrm{A}}\langle 0|_{\mathrm{A}} \otimes I + |1\rangle_{\mathrm{A}}\langle 1|_{\mathrm{A}} \otimes U'(\boldsymbol{\theta'})\right)|\psi_1\rangle \\
&\approx e^{i\alpha}\frac{1}{\sqrt{2}}\sum_{j=0}^{\frac{M}{2}-1}\sqrt{b_j}\left(|0\rangle_{\mathrm{A}}|\boldsymbol{x}_j\rangle_{\mathrm{D}} + e^{-i\phi}|1\rangle_{\mathrm{A}}|\tilde{\boldsymbol{x}}\rangle_{\mathrm{D}}\right)|j\rangle_{\mathrm{J}} \\
&= e^{i\alpha}|\psi_{\mathrm{init}}\rangle . \qquad (25)
\end{aligned}$$

Finally, we operate H on the ancilla qubit to obtain $|\psi_f\rangle$:

$$H^{(\mathrm{A})}e^{i\alpha}|\psi_{\mathrm{init}}\rangle = e^{i\alpha}|\psi_f\rangle . \qquad (26)$$

Although the global phase $e^{i\alpha}$ is added to $|\psi_f\rangle$, this does not affect the probability $\Pr(0)$ and the expectation value $\langle\sigma_z^{(\mathrm{A})}\rangle$. Recall that, $U(\boldsymbol{\theta})$ and $U'(\boldsymbol{\theta'})$ consist of $(n+m+1)$ qubits with $\mathcal{O}(1) \sim \mathcal{O}(\mathrm{poly}(n+m+1))$ layers and n qubits with $\mathcal{O}(1) \sim \mathcal{O}(\mathrm{poly}(n))$ layers, respectively. Therefore, we can implement the approximated CHC without using exponentially many gates.

Before moving forward, we give some remarks, regarding the design of $U'(\boldsymbol{\theta'})$. First, we assume that $U'(\boldsymbol{\theta'})$ consists of only R_y gates and CNOT gates, because $\tilde{\boldsymbol{x}}$ is a real-valued data vector. As a result, the unitary process is restricted to $U'(\boldsymbol{\theta'})|0\rangle_{\mathrm{D}} \approx \pm|\tilde{\boldsymbol{x}}\rangle_{\mathrm{D}}$, which is further converted to $U'(\boldsymbol{\theta'})|0\rangle_{\mathrm{D}} \approx |\tilde{\boldsymbol{x}}\rangle_{\mathrm{D}}$ by compensating the phase via the choice of ϕ in Eq. (25). Next, to implement the controlled $U'(\boldsymbol{\theta'})$, every elementary gate contained in $U'(\boldsymbol{\theta'})$ has to be modified to a controlled gate via the A qubit. Finally, in practice, we construct $U(\boldsymbol{\theta})$ off-line using all the training data; once a test datum $\tilde{\boldsymbol{x}}$ to be classified is given to us, we learn $U'(\boldsymbol{\theta'})$ to approximately encode $\tilde{\boldsymbol{x}}$ and then construct the CHC to predict the corresponding label $\tilde{y}$.

IV. DEMONSTRATION

This section gives two numerical demonstrations to show the performance of our algorithm composed of ACAE and the CHC. First we present an example application to a classification problem for the Iris dataset [39]. Next we show application to the fraud detection problem using the credit card fraud dataset [40] provided in Kaggle.

A. Iris dataset classification

The Iris dataset consists of three iris species (*Iris setosa*, *Iris virginica*, and *Iris versicolor*) with 50 samples each as well as four features (sepal length, sepal width, petal length, and petal width) for each flower. Each sample data includes the identification (ID) number, four features, and the species. IDs for 1 to 50, 51 to 100, and 101 to 150 represent data for *I. setosa*, *I. virginica*, and *I. versicolor*, respectively.

In this paper, we consider the *I. setosa* and *I. versicolor* classification problem. The goal is to create a binary classifier that predicts the correct label $\tilde{y}$ (0 : *I. setosa*, 1 : *I. versicolor*) for the given test data $\tilde{\boldsymbol{x}}$ = (sepal length, sepal width, petal length, petal width). In this demonstration, we employ the first four data of each species as training data. That is, we use data with IDs 1 to 4 and 51 to 54 as training data for *I. setosa* and *I. versicolor*, respectively. On the other hand, we use data with IDs 5 to 8 and 55 to 58 as test data.

First, we need to prepare the quantum state $|\psi_{\mathrm{init}}\rangle$ given in Eq. (18) using ACAE. Since the dimension of the feature vector and the number of training data are $N = 4$ and $M = 8$, the number of required qubits are $n = \lceil\log_2(N)\rceil = 2$ and $m = \lceil\log_2(M/2)\rceil = 2$, respectively. The number of total qubits required for composing the

TABLE I. The data contents embedded in the quantum amplitude of $|\psi_{\text{init}}\rangle$ in *Iris setosa* and *Iris versicolor* classification problem. The A, J, and D in the left three columns represent the ancilla qubit, the qubits for data numbering, and the qubits for data encoding, respectively, as in Fig. 3. The "i" in the right end column represents an imaginary unit. The amplitudes are normalized to satisfy Eq. (16).

A Ancilla qubit 0 : training data 1 : test data	J Qubits for data numbering (from 0 to $M/2$)	D Qubits for data enconding (from 0 to M)	Basis	Data contents embedded in the amplitude of each basis
0	00	00	$\|00000\rangle$	(1st feature of *I. setosa*) + (1st feature of *I. versicolor*)$\times i$
		01	$\|00001\rangle$	(2nd feature of *I. setosa*) + (2nd feature of *I. versicolor*)$\times i$
		10	$\|00010\rangle$	(3rd feature of *I. setosa*) + (3rd feature of *I. versicolor*)$\times i$
		11	$\|00011\rangle$	(4th feature of *I. setosa*) + (4th feature of *I. versicolor*)$\times i$
	$\vdots$	$\vdots$	$\vdots$	$\vdots$
	11	00	$\|01100\rangle$	(1st feature of *I. setosa*) + (1st feature of *I. versicolor*)$\times i$
		01	$\|01101\rangle$	(2nd feature of *I. setosa*) + (2nd feature of *I. versicolor*)$\times i$
		10	$\|01110\rangle$	(3rd feature of *I. setosa*) + (3rd feature of *I. versicolor*)$\times i$
		11	$\|01111\rangle$	(4th feature of *I. setosa*) + (4th feature of *I. versicolor*)$\times i$
1	00	00	$\|10000\rangle$	(1st feature of the test data)$\times \exp(-i\phi)$
		01	$\|10001\rangle$	(2nd feature of the test data)$\times \exp(-i\phi)$
		10	$\|10010\rangle$	(3rd feature of the test data)$\times \exp(-i\phi)$
		11	$\|10011\rangle$	(4th feature of the test data)$\times \exp(-i\phi)$
	$\vdots$	$\vdots$	$\vdots$	$\vdots$
	11	00	$\|11100\rangle$	(1st feature of the test data)$\times \exp(-i\phi)$
		01	$\|11101\rangle$	(2nd feature of the test data)$\times \exp(-i\phi)$
		10	$\|11110\rangle$	(3rd feature of the test data)$\times \exp(-i\phi)$
		11	$\|11111\rangle$	(4th feature of the test data)$\times \exp(-i\phi)$

quantum circuit is $n+m+1 = 5$, which means that $|\psi_{\text{init}}\rangle$ has $2^5 = 32$ amplitudes. Table I shows the data contents embedded in the quantum amplitude of each basis.

We encode the training data of the *I. setosa* and *I. versicolor* into the complex amplitude of the ancilla qubit state $|0\rangle$ by using the PQC $U(\boldsymbol{\theta})$. Also, we encode the test data into the amplitude of the ancilla qubit state $|1\rangle$ by using $U'(\boldsymbol{\theta'})$. We use the 5-qubit and 12-layer ansatz $U(\boldsymbol{\theta})$ illustrated in Fig. 1. We randomly initialize all the directions of each rotating gate (i.e., X, Y, or Z) and θ_r at the beginning of each training. As the optimizer, Adam [41] is used. The number of iterations (i.e., the number of the updates of the parameters) is set to 400 for training $U(\boldsymbol{\theta})$. For each iteration, 1000 classical snapshots are used to estimate the fidelity; that is, we set $N_{\text{shot}} = 1000$ in Eq. (8). The learning rate is 0.1 for the first 100 iterations, 0.01 for the next 100 iterations, 0.005 for the next 100 iterations, and 0.001 for the last 100 iterations. In Fig. 5, we show the change of fidelity in the training process of $U(\boldsymbol{\theta})$. At the end of the training, the fidelity reaches 0.994.

As for the encoding process of test data, we use the 2-qubit and 2-layer ansatz $U'(\boldsymbol{\theta'})$ that consists of only R_y gates and CNOT gates. The number of iterations and N_{shot} are 100 and 1000, respectively. The learning rate is 0.1 for the first 50 iterations and 0.01 for the next 50. We found that, for each test datum, the fidelity reaches the value bigger than 0.999. It is notable that the number of layers of $U'(\boldsymbol{\theta'})$ and the number of iterations for training it are much smaller compared to the encoding process for training data. This means, once the training data are encoded, it is easy to change the test data.

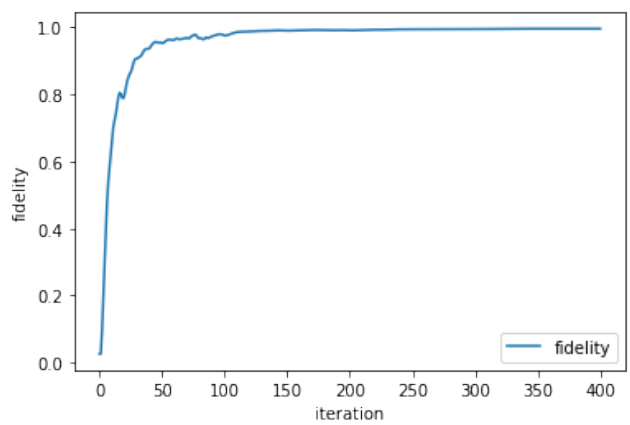
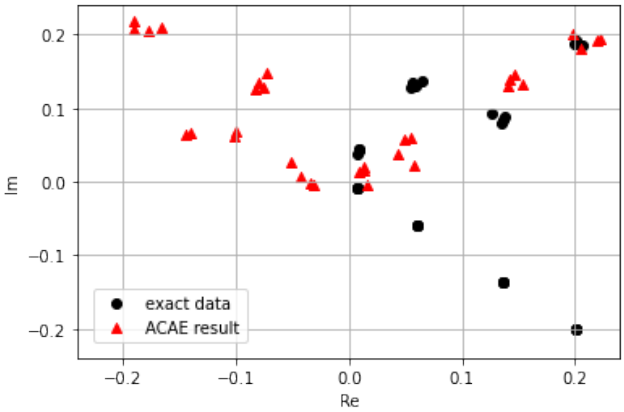
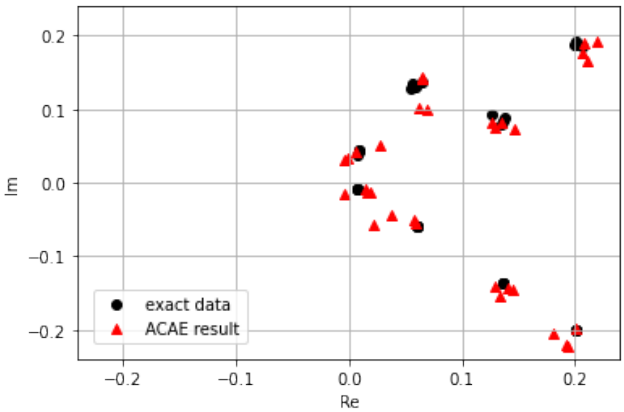


FIG. 5. The change of the fidelity between the target state and the model state in the training process of $U(\boldsymbol{\theta})$. Here, the target state is Eq. (24), and the model state means the state that is actually generated by the PQC $U(\boldsymbol{\theta})$.

As an example of the data encoding results, a set of complex amplitudes generated by ACAE is shown in Fig. 6. In the figure, the value of each complex amplitude is plotted on the complex plane. The black dots and red triangles represent the exact data and the approximate complex amplitudes embedded by ACAE, respectively. Note that the complex amplitude embedded by ACAE



(a)



(b)

FIG. 6. An example of data encoding results. Note that each data value is divided by a constant number to satisfy the normalization condition (16).
(a) The complex amplitudes of $|\psi_{\text{init}}\rangle$ generated by ACAE (red triangles) and exact data (black dots). Note that red dots contain the influence of the global phase $e^{i\alpha}$.
(b) The result in which the global phase is hypothetically eliminated is shown. The fidelity between the model state (red triangles) and the target state (black dots) is 0.993.

contains the global phase $e^{i\alpha}$ as in Eq. (25). In order to compare the exact data with the ACAE result visually, the result in which the global phase is hypothetically eliminated is also shown. Recall that the global phase does not affect the measurement in the remaining procedure.

After the state preparation, we operate the Hadamard gate on the ancilla qubit and obtain $|\psi_f\rangle$ in Eq. (19). By measuring the ancilla qubit of this quantum state and obtaining the sign of $\langle\sigma_z^{(\mathrm{A})}\rangle$, we can use Eq. (22) to predict the label $\tilde{y}$ of the test data $\tilde{\boldsymbol{x}}$, i.e., $\tilde{y} = \mathrm{sgn}\left[\langle\sigma_z^{(\mathrm{A})}\rangle\right]$. Classification results are shown in Table II(a) for 8 cases in which the test data are IDs 5 to 8 and IDs 55 to 58. In addition to the *I. setosa* and *I. versicolor* classification problem, we carry out the *I. versicolor* and *I. virginica* classification problems in the same way and show the results in Table II(b). All classification results are correct in Table II(a). On the other hand, three out of eight classification results are incorrect in Table II(b).

Let us discuss the results. Pairwise relationships in the Iris dataset shows that *I. setosa* can be clearly separated from the other two varieties, whereas the features of *I. versicolor* and *I. virginica* slightly overlap with each other. Therefore, classification of *I. versicolor* and *I. virginica* [Table II(b)] is considered more difficult than that of *I. setosa* and *I. versicolor* [Table II(a)]. This could be the cause of the low accuracy rate of Table II(b). Note that the error between the exact data and the ACAE data also affects the classification accuracy. If we adjust the number of layers in the PQC and the number of iterations in the training process to improve the fidelity between the target state and the model state, we will be able to increase the accuracy rate. In fact, we have confirmed that the incorrect results (#55, #56, #57) turned correct when the classifications were performed using the exact data. Table II(c) shows the results where both the training and the test data are ideally encoded without errors.

B. Credit card fraud detection

Nowadays, credit card fraud is a social problem in terms of customer protection, financial crime prevention, and avoiding negative impacts on corporate finances. The losses that arise from credit card fraud are a serious problem for financial institutions; according to the Nilson Report [42], credit card fraud losses are expected to reach \$49.3 billion by 2030. Banks and credit card companies that pay for fraud will be hit hard by these rising costs. With digital crime and online fraud on the rise, it is more important than ever for financial institutions to prevent credit card fraud through advanced technology and strong security measures. To detect fraudulent use, financial institutions use human judgment to determine fraud based on information such as cardholder attributes, past transactions, and product delivery address information; however, this method requires human resources and costs. Although attempts to detect fraud by machine learning based on features extracted from credit card transaction data also have been attracting attention in recent years, there are disadvantages such as more time and epochs to converge for a stable prediction, excessive training and so on. Quantum machine learning has the potential to solve these challenges, and its application to credit card fraud detection deserves exploring.

In this subsection, we demonstrate credit card fraud detection as another practical application of the CHC with ACAE. The goal is to encode credit card transaction data into a quantum state as training data and classify whether a given transaction datum $\tilde{\boldsymbol{x}}$ is a normal ($\tilde{y} = +1$) or fraudulent transaction ($\tilde{y} = -1$).

In this demonstration we use the credit card fraud de-

TABLE II. The results of the Iris dataset classification problem

(a) *Iris setosa* versus *Iris versicolor*

Test data ID	Class	$\langle\sigma_z^{(\mathrm{A})}\rangle$	$\tilde{y}$	Result
#5		0.0422	+	Correct
#6	*I. setosa*	0.0364	+	Correct
#7	(+)	0.0395	+	Correct
#8		0.0374	+	Correct
#55		-0.0317	−	Correct
#56	*I. versicolor*	-0.0315	−	Correct
#57	(−)	-0.0285	−	Correct
#58		-0.0239	−	Correct

(b) *Iris versicolor* versus *Iris virginica*

Test data ID	Class	$\langle\sigma_z^{(\mathrm{A})}\rangle$	$\tilde{y}$	Result
#55		-0.0001	−	Incorrect
#56	*I. versicolor*	-0.0015	−	Incorrect
#57	(+)	-0.0020	−	Incorrect
#58		0.0010	+	Correct
#105		-0.0083	−	Correct
#106	*I. virginica*	-0.0066	−	Correct
#107	(−)	-0.0089	−	Correct
#108		-0.0054	−	Correct

(c) *Iris versicolor* versus *Iris virginica*(Exact data are used.)

Test data ID	Class	$\langle\sigma_z^{(\mathrm{A})}\rangle$	$\tilde{y}$	Result
#55		0.0322	+	Correct
#56	*I. versicolor*	0.0013	+	Correct
#57	(+)	0.0230	+	Correct
#58		0.0534	+	Correct
#105		-0.0392	−	Correct
#106	*I. virginica*	-0.0269	−	Correct
#107	(−)	-0.0430	−	Correct
#108		-0.0194	−	Correct

tection dataset [40] provided by Kaggle. The dataset contains credit card transactions made by European cardholders in September 2013. The dataset has 284,807 transactions which include 492 fraudulent transactions. Note that, since it is difficult to encode all transaction data into a quantum state, for proof-of-concept testing, we take 4 normal transaction data[1] and 4 fraudulent transaction data[2] from this dataset as training data and perform classification tests to determine whether the given test data are normal or fraudulent. Each set of data consists of *Times*, *Amount,* and 28 different features $(V1, V2, \ldots, V28)$ transformed by principal component analysis. We use 4 features $(V1, V2, V3, V4)$ out of the 28 features for classification, which means that the dimension of the feature vector and the number of training data are $N = 4$ and $M = 8$, respectively, and the number of total qubits required for composing the quantum circuit is $n + m + 1 = 5$. As in the previous subsection, we embed the training data of normal transactions and fraudulent transactions into the real and imaginary parts of the complex amplitude of the ancilla qubit state $|0\rangle$ and embed the test data into the complex amplitude of the ancilla qubit state $|1\rangle$.

After the state preparation, we operate the Hadamard gate on the ancilla qubit and make measurements to obtain $\langle\sigma_z^{(\mathrm{A})}\rangle$, the sign of which provides the classification result, i.e., $\tilde{y} = +1$ for normal data and $\tilde{y} = -1$ for fraudulent data. As test data, we take the top four normal data IDs and fraudulent data IDs, in ascending order, excluding the training data. Classification results are shown in Table III.

TABLE III. The results of fraud detection.

Test data ID[a]	Class	$\langle\sigma_z^{(\mathrm{A})}\rangle$	$\tilde{y}$	Result
#5		0.0699	+	Correct
#6	Normal	0.1978	+	Correct
#7	(+)	0.0183	+	Correct
#8		0.1579	+	Correct
#6330		-0.3747	−	Correct
#6332	Fraud	-0.4245	−	Correct
#6335	(−)	-0.4289	−	Correct
#6337		-0.4163	−	Correct

[a] As test data, we take the top four normal data IDs and fraudulent data IDs, in ascending order, excluding the training data.

Note that the instances can be arbitrarily chosen, rather than the top 4 normal and fraudulent data IDs. Hence, we have conducted an additional simulation with randomly selected 48 instances chosen from the same Kaggle dataset described above. As a result, we have confirmed that 42 out of the 48 test data are correctly classified. Although the remaining 6 test data are incorrectly classified, this misclassification may be caused by the intrinsic characteristics of the dataset and/or an insufficient amount of training data rather than the encoding errors.

V. CONCLUSION AND DISCUSSION

In this paper, we proposed the approximate complex amplitude encoding algorithm (ACAE) which allows for the efficient encoding of given complex-valued classical data into quantum states using shallow parametrized quantum circuits. The key idea of this algorithm is to use the fidelity as a cost function, which can reflect the difference in complex amplitude between the model state and the target state, unlike the MMD-based cost function in the original AAE. Also note that the classical shadow with random Clifford unitary is used for efficient fidelity estimation. In addition, we applied ACAE to realize the CHC with fewer gates than the original CHC

[1] We take the top 4 normal data IDs in ascending order, specifically, #1, #2, #3, and #4.

[2] We take the top 4 fraudulent data IDs in ascending order, specifically, #524, #624, #4921, and #6109.

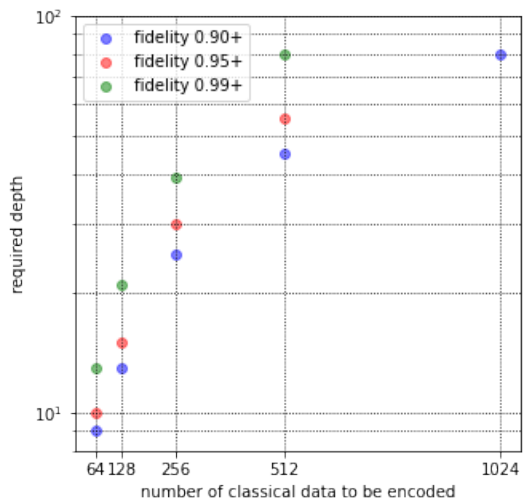


FIG. 7. The required circuit depth to prepare states that reach the fidelity bigger than 0.90, 0.95, or 0.99. For the case of 1024 classical data, we were able to calculate the depth only for the case of fidelity bigger than 0.90, due to the limited computation resources.

which requires an exponential number of gates to prepare the exact quantum state. Using this algorithm we demonstrated the Iris data classification and the credit card fraud detection that is considered as a key challenge in financial institutions.

The main concern in the use of ACAE is its scalability. To discuss this problem, we conducted a numerical simulation for the same credit card fraud detection problem as before [40], to see the relationship between the number of data (accordingly the number of qubits) and the circuit depth required to achieve a particular value of fidelity. Here we examine the case where the number of data varies from 64 (5 qubits) to 1024 (9 qubits). Figure 7 shows that the required depth of the variational quantum circuit increases superlinearly with respect to the number of qubits, implying that ACAE may be not applicable to large-size problems. However, we found that, for the case of 5 qubits, even when the fidelity is decreased from 0.99+ to approximately 0.7 by reducing the number of training steps, the classification accuracy is still above 80%. That is, ACAE may work for some practical problems such that 80% classification accuracy is enough.

For practical applications of ACAE, we need to deal with enormously large datasets. For example, in the demonstration for credit card fraud detection, approximately 280,000 training data are provided and each data has 28 different features. This means $n + m + 1 = \lceil \log_2(28) \rceil + \lceil \log_2(280,000/2) \rceil + 1 = 23$ qubits are required to deal with all the data. As the number of qubits is increased, the degree of freedom of the quantum state exponentially grows; in such a case, there appear several practical problems to be resolved. For example, ACAE employs the fidelity as a cost function, which is, however, known as a global cost that leads to the so-called gradient vanishing problem or the barren plateau problem [43]; i.e., the gradient vector of the cost decays exponentially fast with respect to the number of qubits, and thus the learning process becomes completely stuck for large-size systems. To mitigate this problem, recently the localized fidelity measurement has been proposed in References [44–46]. Moreover, applications of several existing methods such as circuit initialization [47, 48], special structured ansatz [49, 50], and parameter embedding [51] are worth investigating to address the gradient vanishing problem. Another problem from a different perspective is that the random Clifford measurement for producing the classical shadow can be challenging to implement in practice, because $\mathcal{O}\left(n^2/\log_2(n)\right)$ entangling gates are needed to sample from n-qubit Clifford unitaries. References [52, 53] have presented that the Clifford circuit depth over unrestricted architectures is upper bounded by $2n + \mathcal{O}\left(\log_2^2(n)\right)$ for all practical purposes, which may improve the implementation of the fidelity estimation process. Overall, algorithm improvements to deal with these problems are all important and remain as future works.

ACKNOWLEDGMENTS

This work was supported by Grant-in-Aid for JSPS Research Fellow No. 22J01501 and MEXT Quantum Leap Flagship Program Grants No. JPMXS0118067285 and No. JPMXS0120319794. We acknowledge the use of IBM Quantum services for this work. We specially thank Kohei Oshio, Ryo Nagai, Ruho Kondo, Yuki Sato, and Dmitri Maslov for their comments on the early version of this draft. We are also grateful to Shumpei Uno for helpful support and discussions on the early stage of this study.

The views expressed are those of the authors and do not reflect the official policy or position of IBM or the IBM Quantum team.

[1] P. Wittek. Quantum machine learning: what quantum computing means to data mining. Academic Press, 2014.

[2] J. Biamonte, P. Wittek, N. Pancotti, P. Rebentrost, N. Wiebe and S. Lloyd. Quantum machine learning. Nature, 549(7671):195–202, 2017.

[3] M. Schuld and F. Petruccione. Supervised learning with quantum computers, volume 17. Springer, 2018.

[4] V. Dunjko and H. J. Briegel. Machine learning & artificial intelligence in the quantum domain: a review of recent progress. Reports on Progress in Physics, 81(7):074001,

2018.

[5] C. Ciliberto, M. Herbster, A. D. Ialongo, M. Pontil, A. Rocchetto, S. Severini and L. Wossnig. Quantum machine learning: a classical perspective. Proceedings of the Royal Society A: Mathematical, Physical and Engineering Sciences, 474(2209):20170551, 2018.

[6] M. Schuld. Supervised quantum machine learning models are kernel methods. arXiv preprint arXiv:2101.11020, 2021.

[7] V. Giovannetti, S. Lloyd and L. Maccone. Quantum random access memory. Physical review letters, 100(16):160501, 2008.

[8] D. K. Park, F. Petruccione and J.-K. K. Rhee. Circuit-based quantum random access memory for classical data. Scientific reports, 9(1):1–8, 2019.

[9] T. M. De Veras, I. C. De Araujo, D. K. Park and A. J. Da Silva. Circuit-based quantum random access memory for classical data with continuous amplitudes. IEEE Transactions on Computers, 70(12):2125–2135, 2020.

[10] A. W. Harrow, A. Hassidim and S. Lloyd. Quantum algorithm for linear systems of equations. Physical review letters, 103(15):150502, 2009.

[11] P. Rebentrost, M. Mohseni and S. Lloyd. Quantum support vector machine for big data classification. Physical review letters, 113(13):130503, 2014.

[12] S. Lloyd, M. Mohseni and P. Rebentrost. Quantum principal component analysis. Nature Physics, 10(9):631–633, 2014.

[13] C. Blank, D. K. Park, J.-K. K. Rhee and F. Petruccione. Quantum classifier with tailored quantum kernel. npj Quantum Information, 6(1):1–7, 2020.

[14] M. Schuld, M. Fingerhuth and F. Petruccione. Implementing a distance-based classifier with a quantum interference circuit. EPL (Europhysics Letters), 119(6):60002, 2017.

[15] C. Blank, A. J. da Silva, L. P. de Albuquerque, F. Petruccione and D. K. Park. Compact quantum kernel-based binary classifier. Quantum Science and Technology, 7(4):045007, 2022.

[16] D. K. Park, C. Blank and F. Petruccione. The theory of the quantum kernel-based binary classifier. Physics Letters A, 384(21):126422, 2020.

[17] L. K. Grover. Synthesis of quantum superpositions by quantum computation. Physical review letters, 85(6):1334, 2000.

[18] Y. R. Sanders, G. H. Low, A. Scherer and D. W. Berry. Black-box quantum state preparation without arithmetic. Physical review letters, 122(2):020502, 2019.

[19] M. Plesch and Č. Brukner. Quantum-state preparation with universal gate decompositions. Physical Review A, 83(3):032302, 2011.

[20] V. V. Shende, S. S. Bullock and I. L. Markov. Synthesis of quantum logic circuits. In Proceedings of the 2005 Asia and South Pacific Design Automation Conference, pages 272–275, 2005.

[21] L. Grover and T. Rudolph. Creating superpositions that correspond to efficiently integrable probability distributions. arXiv preprint quant-ph/0208112, 2002.

[22] M. Möttönen, J. J. Vartiainen, V. Bergholm and M. M. Salomaa. Quantum circuits for general multiqubit gates. Physical review letters, 93(13):130502, 2004.

[23] V. V. Shende and I. L. Markov. Quantum circuits for incompletely specified two-qubit operators. arXiv preprint quant-ph/0401162, 2004.

[24] K. Nakaji, S. Uno, Y. Suzuki, R. Raymond, T. Onodera, T. Tanaka, H. Tezuka, N. Mitsuda and N. Yamamoto. Approximate amplitude encoding in shallow parameterized quantum circuits and its application to financial market indicators. Phys. Rev. Research, 4:023136, 2022.

[25] P. J. Ollitrault, G. Mazzola and I. Tavernelli. Nonadiabatic molecular quantum dynamics with quantum computers. Physical Review Letters, 125(26):260511, 2020.

[26] H. H. S. Chan, R. Meister, T. Jones, D. P. Tew and S. C. Benjamin. Grid-based methods for chemistry modelling on a quantum computer. arXiv preprint arXiv:2202.05864, 2022.

[27] J.-G. Liu and L. Wang. Differentiable learning of quantum circuit Born machines. Physical Review A, 98(6):062324, 2018.

[28] B. Coyle, D. Mills, V. Danos and E. Kashefi. The Born supremacy: Quantum advantage and training of an ising Born machine. npj Quantum Information, 6(1):1–11, 2020.

[29] H.-Y. Huang, R. Kueng and J. Preskill. Predicting many properties of a quantum system from very few measurements. Nature Physics, 16:1050–1057, 2020.

[30] G. M. D'Ariano, M. G. Paris and M. F. Sacchi. Quantum tomography. Advances in Imaging and Electron Physics, 128:206–309, 2003.

[31] S. Hillmich, C. Hadfield, R. Raymond, A. Mezzacapo and R. Wille. Decision diagrams for quantum measurements with shallow circuits. In 2021 IEEE International Conference on Quantum Computing and Engineering (QCE), pages 24–34. IEEE, 2021.

[32] S. Aaronson and D. Gottesman. Improved simulation of stabilizer circuits. Physical Review A, 70(5):052328, 2004.

[33] K. N. Patel, I. L. Markov and J. P. Hayes. Optimal synthesis of linear reversible circuits. Quantum Inf. Comput., 8(3):282–294, 2008.

[34] A. Kandala, A. Mezzacapo, K. Temme, M. Takita, M. Brink, J. M. Chow and J. M. Gambetta. Hardware-efficient variational quantum eigensolver for small molecules and quantum magnets. Nature, 549(7671):242–246, 2017.

[35] D. Gottesman. The heisenberg representation of quantum computers. arXiv preprint quant-ph/9807006, 1998.

[36] G. E. Crooks. Gradients of parameterized quantum gates using the parameter-shift rule and gate decomposition. arXiv preprint arXiv:1905.13311, 2019.

[37] M. Mottonen, J. J. Vartiainen, V. Bergholm and M. M. Salomaa. Transformation of quantum states using uniformly controlled rotations. arXiv preprint quant-ph/0407010, 2004.

[38] V. Bergholm, J. J. Vartiainen, M. Möttönen and M. M. Salomaa. Quantum circuits with uniformly controlled one-qubit gates. Physical Review A, 71(5):052330, 2005.

[39] Iris species. `https://www.kaggle.com/datasets/uciml/iris`.

[40] Credit card fraud detection. `https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud`.

[41] D. P. Kingma and J. Ba. Adam: A method for stochastic optimization. arXiv:1412.6980, 2014.

[42] Nilson report, 2021. `https://nilsonreport.com/upload/content_promo/NilsonReport_Issue1209.pdf`.

[43] J. R. McClean, S. Boixo, V. N. Smelyanskiy, R. Babbush and H. Neven. Barren plateaus in quantum neural network training landscapes. Nature communications,

9(1):1–6, 2018.

[44] S. Khatri, R. LaRose, A. Poremba, L. Cincio, A. T. Sornborger and P. J. Coles. Quantum-assisted quantum compiling. Quantum, 3:140, 2019.

[45] K. Sharma, S. Khatri, M. Cerezo and P. J. Coles. Noise resilience of variational quantum compiling. New Journal of Physics, 22(4):043006, 2020.

[46] H. Tezuka, S. Uno and N. Yamamoto. Generative model for learning quantum ensemble via optimal transport loss. arXiv preprint arXiv:2210.10743, 2022.

[47] E. Grant, L. Wossnig, M. Ostaszewski and M. Benedetti. An initialization strategy for addressing barren plateaus in parametrized quantum circuits. Quantum, 3:214, 2019.

[48] K. Zhang, M.-H. Hsieh, L. Liu and D. Tao. Gaussian initializations help deep variational quantum circuits escape from the barren plateau. arXiv preprint arXiv:2203.09376, 2022.

[49] M. Cerezo, A. Sone, T. Volkoff, L. Cincio and P. J. Coles. Cost function dependent barren plateaus in shallow parametrized quantum circuits. Nature communications, 12(1):1–12, 2021.

[50] X. Liu, G. Liu, J. Huang and X. Wang. Mitigating barren plateaus of variational quantum eigensolvers. arXiv preprint arXiv:2205.13539, 2022.

[51] T. Volkoff and P. J. Coles. Large gradients via correlation in random parameterized quantum circuits. Quantum Science and Technology, 6(2):025008, 2021.

[52] D. Maslov and B. Zindorf. Depth optimization of CZ, CNOT, and Clifford circuits. IEEE Transactions on Quantum Engineering, 2022.

[53] D. Maslov and W. Yang. CNOT circuits need little help to implement arbitrary hadamard-free clifford transformations they generate, 2022.